

CS787: GENERATIVE ARTIFICIAL INTELLIGENCE ARTIFICIAL NEURAL NETWORKS

Arnab Bhattacharya
arnabb@cse.iitk.ac.in

Computer Science and Engineering,
Indian Institute of Technology, Kanpur
<http://web.cse.iitk.ac.in/~cs787/>

1st semester, 2026-27

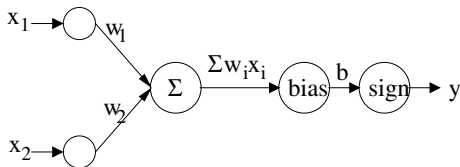
Wed 12:00–13:15 at L20, Fri 12:00–13:15 at L19

Perceptron

- A **perceptron** is a simple binary *linear* classifier
- Input attributes $x_1, \dots, x_n$ are weighted and summed
- A bias b is added as well
- Final class ($y = \pm 1$) is *sign* of the output
- The *sign* node is called the **activation function**

Perceptron

- A **perceptron** is a simple binary *linear* classifier
- Input attributes $x_1, \dots, x_n$ are weighted and summed
- A bias b is added as well
- Final class ($y = \pm 1$) is *sign* of the output
- The *sign* node is called the **activation function**
- It is a simple *linear* classifier
- Decision boundary is $w \cdot x + b = \sum_{i=1}^n w_i x_i + b$
- Therefore, **sign** of $w \cdot x + b$ predicts the class

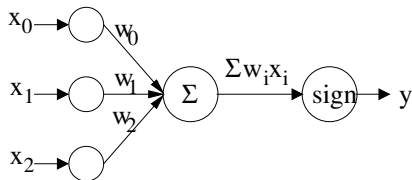


- Why is bias needed?

Bias

- Why is bias needed?
- Otherwise, hyperplane passes through origin
- So, bias is treated differently

- Why is bias needed?
- Otherwise, hyperplane passes through origin
- So, bias is treated differently
- It can be treated uniformly as well: include 1 as x_0 of data
- Decision boundary becomes $w_0x_0 + w.x = \sum_{i=0}^n w_ix_i$
- Weight on $x_0 = 1$ becomes the constant term, i.e., $w_0 = b$



Examples of Perceptrons

- Different Boolean functions
- AND (of x_1 and x_2)

Examples of Perceptrons

- Different Boolean functions
- AND (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -1.5$
- OR (of x_1 and x_2)

Examples of Perceptrons

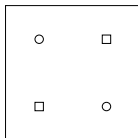
- Different Boolean functions
- AND (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -1.5$
- OR (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -0.5$
- NOT (of x_1)

Examples of Perceptrons

- Different Boolean functions
- AND (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -1.5$
- OR (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -0.5$
- NOT (of x_1)
 - $w_1 = -1, w_0 = 0.5$
- XOR (of x_1 and x_2)

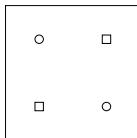
Examples of Perceptrons

- Different Boolean functions
- AND (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -1.5$
- OR (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -0.5$
- NOT (of x_1)
 - $w_1 = -1, w_0 = 0.5$
- XOR (of x_1 and x_2)
 - Cannot be done as the two classes are not linearly separable



Examples of Perceptrons

- Different Boolean functions
- AND (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -1.5$
- OR (of x_1 and x_2)
 - $w_1 = w_2 = 1, w_0 = -0.5$
- NOT (of x_1)
 - $w_1 = -1, w_0 = 0.5$
- XOR (of x_1 and x_2)
 - Cannot be done as the two classes are not linearly separable



- Combination of perceptrons can do it, though

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- Perceptron learning rule or perceptron training rule

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

- If $y_i = \hat{y}_i$, there is no change in weight

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

- If $y_i = \hat{y}_i$, there is no change in weight
- If $y_i = +1$ and $\hat{y}_i = -1$, i.e., $\hat{y}_i - y_i < 0$
 - Weights of positive x_i are increased and those of negative x_i are decreased thereby pushing $\hat{y}_i$ towards positive

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

- If $y_i = \hat{y}_i$, there is no change in weight
- If $y_i = +1$ and $\hat{y}_i = -1$, i.e., $\hat{y}_i - y_i < 0$
 - Weights of positive x_i are increased and those of negative x_i are decreased thereby pushing $\hat{y}_i$ towards positive
- If $y_i = -1$ and $\hat{y}_i = +1$, i.e., $\hat{y}_i - y_i > 0$
 - Weights of positive x_i are decreased and those of negative x_i are increased thereby pushing $\hat{y}_i$ towards negative

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

- If $y_i = \hat{y}_i$, there is no change in weight
- If $y_i = +1$ and $\hat{y}_i = -1$, i.e., $\hat{y}_i - y_i < 0$
 - Weights of positive x_i are increased and those of negative x_i are decreased thereby pushing $\hat{y}_i$ towards positive
- If $y_i = -1$ and $\hat{y}_i = +1$, i.e., $\hat{y}_i - y_i > 0$
 - Weights of positive x_i are decreased and those of negative x_i are increased thereby pushing $\hat{y}_i$ towards negative
- If the data *is* linearly separable, a perceptron *will* learn it, i.e., it will converge to the global optimum; otherwise, it may oscillate

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

- If $y_i = \hat{y}_i$, there is no change in weight
- If $y_i = +1$ and $\hat{y}_i = -1$, i.e., $\hat{y}_i - y_i < 0$
 - Weights of positive x_i are increased and those of negative x_i are decreased thereby pushing $\hat{y}_i$ towards positive
- If $y_i = -1$ and $\hat{y}_i = +1$, i.e., $\hat{y}_i - y_i > 0$
 - Weights of positive x_i are decreased and those of negative x_i are increased thereby pushing $\hat{y}_i$ towards negative
- If the data *is* linearly separable, a perceptron *will* learn it, i.e., it will converge to the global optimum; otherwise, it may oscillate
- The learning rates η may be modified to give more importance to recent examples

Learning a Perceptron

- What is new in a linear classifier?
- Training a perceptron, i.e., learning the weights w
- **Perceptron learning rule** or **perceptron training rule**

$$w_i^{new} = w_i^{old} - \eta(\hat{y}_i - y_i)x_i$$

where $\hat{y}_i$ is the predicted value and η is the *learning rate*

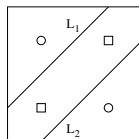
- If $y_i = \hat{y}_i$, there is no change in weight
- If $y_i = +1$ and $\hat{y}_i = -1$, i.e., $\hat{y}_i - y_i < 0$
 - Weights of positive x_i are increased and those of negative x_i are decreased thereby pushing $\hat{y}_i$ towards positive
- If $y_i = -1$ and $\hat{y}_i = +1$, i.e., $\hat{y}_i - y_i > 0$
 - Weights of positive x_i are decreased and those of negative x_i are increased thereby pushing $\hat{y}_i$ towards negative
- If the data *is* linearly separable, a perceptron *will* learn it, i.e., it will converge to the global optimum; otherwise, it may oscillate
- The learning rates η may be modified to give more importance to recent examples
- Weights can be learned through *gradient descent* method as well

Combination of Perceptrons

- A single perceptron can learn only a single hyperplane
- If the data can be separated using two or more hyperplanes, a *combination* of perceptrons can learn it
- Example: XOR

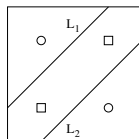
Combination of Perceptrons

- A single perceptron can learn only a single hyperplane
- If the data can be separated using two or more hyperplanes, a *combination* of perceptrons can learn it
- Example: XOR

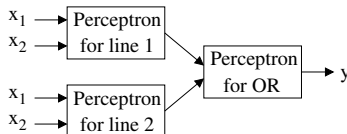


Combination of Perceptrons

- A single perceptron can learn only a single hyperplane
- If the data can be separated using two or more hyperplanes, a *combination* of perceptrons can learn it
- Example: XOR



- Each hyperplane can be modeled by a perceptron and the outputs can be combined using OR

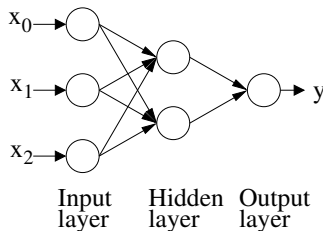


Artificial Neural Networks

- **Artificial neural networks (ANNs)** are modeled on the human brain
- Nodes have connections ala neurons in human nervous system
- Nodes are also called **neurodes**

Artificial Neural Networks

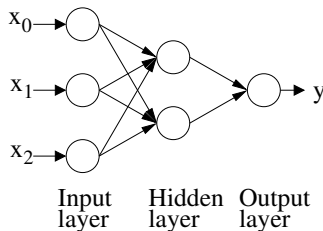
- **Artificial neural networks (ANNs)** are modeled on the human brain
- Nodes have connections ala neurons in human nervous system
- Nodes are also called **neurodes**
- Three types of nodes: input layer, *hidden* layer, output layer



- ANN with one hidden layer is considered as *two-layered*
 - Input layer is not counted
- Can have multiple hidden layers – **deep neural network (DNN)**

Artificial Neural Networks

- **Artificial neural networks (ANNs)** are modeled on the human brain
- Nodes have connections ala neurons in human nervous system
- Nodes are also called **neurodes**
- Three types of nodes: input layer, *hidden* layer, output layer



- ANN with one hidden layer is considered as *two-layered*
 - Input layer is not counted
- Can have multiple hidden layers – **deep neural network (DNN)**
- Learning through ANNs is also called **connectionist learning**

Connections

- ANNs are of many types depending on the connections
- The most common are **multilayer feed-forward networks**
 - Connections are directed from one layer *only* to the next
 - There are *no* back edges or same-layer connections

Connections

- ANNs are of many types depending on the connections
- The most common are **multilayer feed-forward networks**
 - Connections are directed from one layer *only* to the next
 - There are *no* back edges or same-layer connections
- In **recurrent networks**, there can be back edges or same-layer connections

Connections

- ANNs are of many types depending on the connections
- The most common are **multilayer feed-forward networks**
 - Connections are directed from one layer *only* to the next
 - There are *no* back edges or same-layer connections
- In **recurrent networks**, there can be back edges or same-layer connections
- *Fully connected*, i.e., each node in one layer is connected to every node in the next layer

Connections

- ANNs are of many types depending on the connections
- The most common are **multilayer feed-forward networks**
 - Connections are directed from one layer *only* to the next
 - There are *no* back edges or same-layer connections
- In **recurrent networks**, there can be back edges or same-layer connections
- *Fully connected*, i.e., each node in one layer is connected to every node in the next layer
- Training is learning the weights on each of these edges
- Akin to layers of perceptrons
- Activation functions in the nodes are *non-linear*

Connections

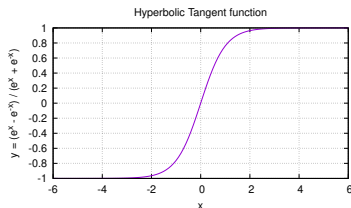
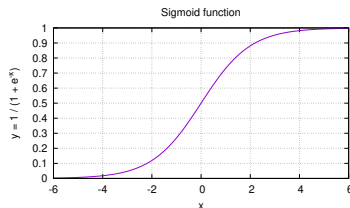
- ANNs are of many types depending on the connections
- The most common are **multilayer feed-forward networks**
 - Connections are directed from one layer *only* to the next
 - There are *no* back edges or same-layer connections
- In **recurrent networks**, there can be back edges or same-layer connections
- *Fully connected*, i.e., each node in one layer is connected to every node in the next layer
- Training is learning the weights on each of these edges
- Akin to layers of perceptrons
- Activation functions in the nodes are *non-linear*
 - Non-linear needed since combination of linear functions can only learn linear separators
- Activation function is **sigmoid (logistic)** or **hyperbolic tangent**

Sigmoid and Hyperbolic Tangent Functions

- If input of a node is x , then output y is

$$y = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$y = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- Approximates the step (or sign) function
- Continuous and differentiable
- Output constrained to $(0, 1)$ or $(-1, +1)$
- Scaled versions of each other
- Also called **squashing functions**

Training an ANN

- Training an ANN requires

Training an ANN

- Training an ANN requires
 - Designing the topology: how many layers and many nodes in each layer

Training an ANN

- Training an ANN requires
 - Designing the topology: how many layers and many nodes in each layer
 - Learning the weights of the connections

Training an ANN

- Training an ANN requires
 - Designing the topology: how many layers and many nodes in each layer
 - Learning the weights of the connections
- Designing the topology requires either extensive domain knowledge or simply try-and-test

Training an ANN

- Training an ANN requires
 - Designing the topology: how many layers and many nodes in each layer
 - Learning the weights of the connections
- Designing the topology requires either extensive domain knowledge or simply try-and-test
- The final outputs are some non-linear functions of inputs
- Hence, can be trained using gradient descent
- However, it is too complex and slow

Training an ANN

- Training an ANN requires
 - Designing the topology: how many layers and many nodes in each layer
 - Learning the weights of the connections
- Designing the topology requires either extensive domain knowledge or simply try-and-test
- The final outputs are some non-linear functions of inputs
- Hence, can be trained using gradient descent
- However, it is too complex and slow
- Weights are updated using through the **backpropagation** algorithm

Nodes and Weights

- Output of a node is the sigmoid of the weighted sum of its inputs
- Inputs, in turn, are outputs of previous layers

Nodes and Weights

- Output of a node is the sigmoid of the weighted sum of its inputs
- Inputs, in turn, are outputs of previous layers
- Inputs are normalized to $(0, 1)$
- Outputs are already constrained to $(0, 1)$

Nodes and Weights

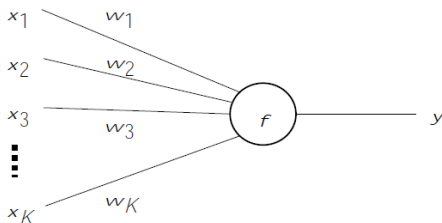
- Output of a node is the sigmoid of the weighted sum of its inputs
- Inputs, in turn, are outputs of previous layers
- Inputs are normalized to $(0, 1)$
- Outputs are already constrained to $(0, 1)$
- Weight from a node i to node j is w_{ij}
- Output from node i is O_i
- Input to node j is I_j

$$I_j = \sum_{\forall i} (w_{ij} \cdot O_i)$$

- Output from node j is O_j

$$O_j = \sigma(I_j) = \frac{1}{1 + e^{-I_j}}$$

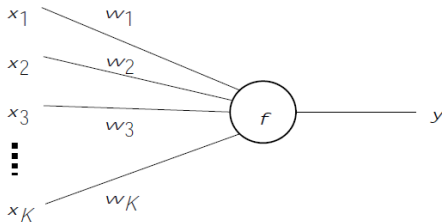
Backpropagation



- Main idea

- Start with arbitrary weights
- Propagate forward the values
- Compute the error (or loss)
- Propagate backward the errors
- Update the weights with respect to error using gradient descent

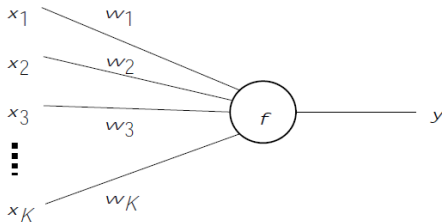
Backpropagation



- Output, using sigmoid function $\sigma(u) = \frac{1}{1+e^{-u}}$, is

$$y = f(u) = \sigma\left(\sum_{\forall x_i} w_i \cdot x_i\right)$$

Backpropagation



- Output, using sigmoid function $\sigma(u) = \frac{1}{1+e^{-u}}$, is

$$y = f(u) = \sigma\left(\sum_{\forall x_i} w_i \cdot x_i\right)$$

- Derivatives of activation functions

$$\frac{d\sigma(u)}{du} = \sigma(u)\sigma(-u) = \sigma(u)(1 - \sigma(u))$$

$$\frac{d(\tanh(z))}{d(z)} = 1 - \tanh^2(z)$$

Updating Weights

- If true output is t , error is squared error

$$E = \frac{1}{2}(y - t)^2$$

Updating Weights

- If true output is t , error is squared error

$$E = \frac{1}{2}(y - t)^2$$

- *Stochastic gradient descent*
- Error function with respect to a single weight w_i

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial w_i} = (y - t) \cdot y(1 - y) \cdot x_i$$

Updating Weights

- If true output is t , error is squared error

$$E = \frac{1}{2}(y - t)^2$$

- *Stochastic gradient descent*
- Error function with respect to a single weight w_i

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial w_i} = (y - t) \cdot y(1 - y) \cdot x_i$$

- Therefore, update equation is

$$w_i^{(new)} = w_i^{(old)} - \eta \cdot (y - t) \cdot y(1 - y) \cdot x_i$$

Updating Weights

- If true output is t , error is squared error

$$E = \frac{1}{2}(y - t)^2$$

- *Stochastic gradient descent*
- Error function with respect to a single weight w_i

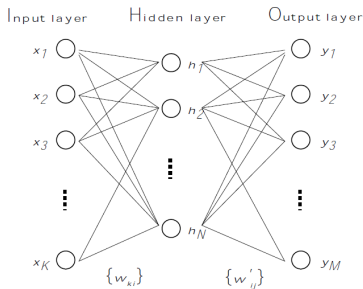
$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial w_i} = (y - t) \cdot y(1 - y) \cdot x_i$$

- Therefore, update equation is

$$w_i^{(new)} = w_i^{(old)} - \eta \cdot (y - t) \cdot y(1 - y) \cdot x_i$$

- **Learning rate** η controls the speed of training
- Can be continued till error is below a threshold

Backpropagation for Multiple Outputs

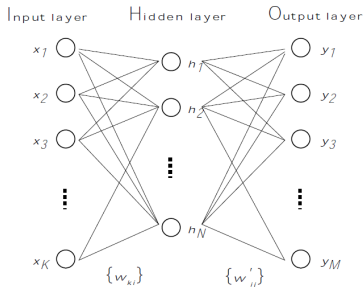


$$h_i = \sigma(u_i) = \sigma\left(\sum_{k=1}^K w_{ki} \cdot x_k\right)$$

$$y_j = \sigma(u'_j) = \sigma\left(\sum_{i=1}^N w'_{ij} \cdot h_i\right)$$

$$E(\vec{x}, \vec{t}, W, W') = \frac{1}{2} \sum_{j=1}^M (y_j - t_j)^2$$

Backpropagation: Output Layer Weights

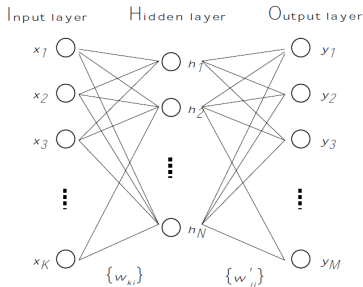


$$\frac{\partial E}{\partial u'_j} = \frac{\partial E}{\partial y_j} \cdot \frac{\partial y_j}{\partial u'_j} = (y_j - t_j) \cdot y_j(1 - y_j) = El'_j$$

$$\frac{\partial E}{\partial w'_{ij}} = \frac{\partial E}{\partial u'_j} \cdot \frac{\partial u'_j}{\partial w'_{ij}} = El'_j \cdot h_i$$

$$w'_{ij}^{(new)} = w'_{ij}^{(old)} - \eta \cdot \frac{\partial E}{\partial w'_{ij}} = w'_{ij}^{(old)} - \eta \cdot El'_j \cdot h_i$$

Backpropagation: Hidden Layer Weights



$$\frac{\partial E}{\partial u_i} = \frac{\partial E}{\partial h_i} \cdot \frac{\partial h_i}{\partial u_i} = \left(\sum_{j=1}^M \frac{\partial E}{\partial u'_j} \cdot \frac{\partial u'_j}{\partial h_i} \right) \cdot \frac{\partial h_i}{\partial u_i} = \sum_{j=1}^M El'_j \cdot w'_{ij} \cdot h_i(1 - h_i) = El_i$$

$$\frac{\partial E}{\partial w_{ki}} = \frac{\partial E}{\partial u_i} \cdot \frac{\partial u_i}{\partial w_{ki}} = El_i \cdot x_k$$

$$w_{ki}^{(new)} = w_{ki}^{(old)} - \eta \cdot \frac{\partial E}{\partial w_{ki}} = w_{ki}^{(old)} - \eta \cdot El_i \cdot x_k$$

Representational Power of an ANN

- Boolean functions

Representational Power of an ANN

- Boolean functions

- Can approximate *any* finite Boolean function with one layer of hidden nodes
- Number of hidden nodes may be equal to exponential factor of number of Boolean variables
- Output layer wired through AND or OR

Representational Power of an ANN

- Boolean functions
 - Can approximate *any* finite Boolean function with one layer of hidden nodes
 - Number of hidden nodes may be equal to exponential factor of number of Boolean variables
 - Output layer wired through AND or OR
- Continuous functions

Representational Power of an ANN

- Boolean functions

- Can approximate *any* finite Boolean function with one layer of hidden nodes
- Number of hidden nodes may be equal to exponential factor of number of Boolean variables
- Output layer wired through AND or OR

- Continuous functions

- Can approximate *any* continuous function with one layer of hidden nodes up to any arbitrary error factor
- Due to properties of sigmoid/tanh functions
- Number of hidden nodes may be large

Representational Power of an ANN

- Boolean functions
 - Can approximate *any* finite Boolean function with one layer of hidden nodes
 - Number of hidden nodes may be equal to exponential factor of number of Boolean variables
 - Output layer wired through AND or OR
- Continuous functions
 - Can approximate *any* continuous function with one layer of hidden nodes up to any arbitrary error factor
 - Due to properties of sigmoid/tanh functions
 - Number of hidden nodes may be large
- Arbitrary functions

Representational Power of an ANN

- Boolean functions

- Can approximate *any* finite Boolean function with one layer of hidden nodes
- Number of hidden nodes may be equal to exponential factor of number of Boolean variables
- Output layer wired through AND or OR

- Continuous functions

- Can approximate *any* continuous function with one layer of hidden nodes up to any arbitrary error factor
- Due to properties of sigmoid/tanh functions
- Number of hidden nodes may be large

- Arbitrary functions

- Can approximate *any* arbitrary function with *two* layers of hidden nodes up to any arbitrary error factor
- Due to properties of sigmoid/tanh functions
- Number of hidden nodes may be large

Discussion

Discussion

- Determining number of nodes and layers is problematic

Discussion

- Determining number of nodes and layers is problematic
- Universal approximators
 - Sigmoid and hyperbolic tangent functions

Discussion

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight

Discussion

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding

Discussion

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only
- Susceptible to class imbalance problem

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only
- Susceptible to class imbalance problem
- Generally, requires large training data

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only
- Susceptible to class imbalance problem
- Generally, requires large training data
- Very slow to train

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only
- Susceptible to class imbalance problem
- Generally, requires large training data
- Very slow to train
- Can be easily parallelized

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only
- Susceptible to class imbalance problem
- Generally, requires large training data
- Very slow to train
- Can be easily parallelized
- Notoriously non-explainable

Discussion

- Determining number of nodes and layers is problematic
- **Universal approximators**
 - Sigmoid and hyperbolic tangent functions
- Redundant or irrelevant features will have less weight
- Does not require explicit feature encoding
- Overfitting is a problem
- Gradient descent converges to local minimum only
- Susceptible to class imbalance problem
- Generally, requires large training data
- Very slow to train
- Can be easily parallelized
- Notoriously non-explainable
- Sensitive to feature values

Stochastic Gradient Descent

- **Stochastic gradient descent** computes the gradients example by example
 - Faster
- **Batch gradient descent** does it for a batch of data
 - More accurate
- It can be the entire training set
- **Epoch** looks at the entire training set
- **Batch** divides the entire training set to manageable chunks

ANN in Matrix Form

- For layer k
 - Input vector $\mathbf{y}^{(k-1)}$ of dimension d_{k-1}
 - Weight matrix $\mathbf{W}^{(k)}$ of dimension $d_{k-1} \times d_k$
 - Bias vector $\mathbf{b}^{(k)}$ of dimension d_k
 - Vector before activation $\mathbf{z}^{(k)}$ of dimension d_k
 - Output vector after activation $\mathbf{y}^{(k)}$ of dimension d_k
 - Activation function g

$$\mathbf{y}^{(k)} = g(\mathbf{z}^{(k)}) \quad \mathbf{z}^{(k)} = \mathbf{W}^{(k)T} \mathbf{y}^{(k-1)} + \mathbf{b}^{(k)}$$

- Input layer is layer 0
- Loss or Error $e(\mathbf{y}^{(L)}, \mathbf{t})$ is at last layer L for true output vector $\mathbf{t}$

ANN in Matrix Form

- For layer k
 - Input vector $\mathbf{y}^{(k-1)}$ of dimension d_{k-1}
 - Weight matrix $\mathbf{W}^{(k)}$ of dimension $d_{k-1} \times d_k$
 - Bias vector $\mathbf{b}^{(k)}$ of dimension d_k
 - Vector before activation $\mathbf{z}^{(k)}$ of dimension d_k
 - Output vector after activation $\mathbf{y}^{(k)}$ of dimension d_k
 - Activation function g

$$\mathbf{y}^{(k)} = g(\mathbf{z}^{(k)}) \quad \mathbf{z}^{(k)} = \mathbf{W}^{(k)T} \mathbf{y}^{(k-1)} + \mathbf{b}^{(k)}$$

- Input layer is layer 0
- Loss or Error $e(\mathbf{y}^{(L)}, \mathbf{t})$ is at last layer L for true output vector $\mathbf{t}$
- For squared loss and sigmoid activation

$$e(\mathbf{y}^{(L)}, \mathbf{t}) = \frac{1}{2} \|\mathbf{y}^{(L)} - \mathbf{t}\|^2 \quad \mathbf{y}^{(k)} = \sigma(\mathbf{z}^{(k)})$$

Backpropagation in Matrix Form: Last Layer

- We need the gradient matrices of error with respect to weight matrices, i.e., $\partial e / \partial W_i$

Backpropagation in Matrix Form: Last Layer

- We need the gradient matrices of error with respect to weight matrices, i.e., $\partial e / \partial W_i$
- Last layer

$$\frac{\partial e}{\partial \mathbf{y}^{(L)}} = \mathbf{e}' \text{ of dimension } d_L \times 1$$

$$\frac{\partial e}{\partial \mathbf{z}^{(L)}} = \frac{\partial e}{\partial \mathbf{y}^{(L)}} \frac{\mathbf{y}^{(L)}}{\mathbf{z}^{(L)}} = \mathbf{e}' \odot \mathbf{g}'(\mathbf{z}^{(L)}) \text{ of dimension } d_L \times 1$$

- Error gradient at each layer k is denoted by $\mathbf{d}^{(k)}$

$$\mathbf{d}^{(k)} = \frac{\partial e}{\partial \mathbf{z}^{(k)}} \text{ of dimension } d_k \times 1$$

- Gradient matrix at last layer

$$\frac{\partial e}{\partial \mathbf{W}^{(L)}} = \frac{\partial e}{\partial \mathbf{y}^{(L)}} \frac{\partial \mathbf{y}^{(L)}}{\partial \mathbf{z}^{(L)}} \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{W}^{(L)}}$$

$$= \mathbf{d}^{(L)} \cdot \mathbf{y}^{(L-1)T} \text{ of dimension } (d_L \times 1) \cdot (1 \times d_{L-1}) = d_L \times d_{L-1}$$

Backpropagation in Matrix Form: Other Layers

- Error gradient at layer $L - 1$

$$\begin{aligned}\mathbf{d}^{(L-1)} &= \frac{\partial e}{\partial \mathbf{z}^{(L-1)}} = \frac{\partial e}{\partial \mathbf{z}^{(L)}} \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{z}^{(L-1)}} \\ &= \mathbf{d}^{(L)} \frac{\partial \mathbf{z}^{(L)}}{\partial \mathbf{y}^{(L-1)}} \frac{\partial \mathbf{y}^{(L-1)}}{\partial \mathbf{z}^{(L-1)}} = \mathbf{W}^{(L)} \cdot \mathbf{d}^{(L)} \odot \mathbf{g}'(\mathbf{z}^{(L-1)})\end{aligned}$$

- In general

$$\mathbf{d}^{(k)} = \mathbf{W}^{(k+1)} \cdot \mathbf{d}^{(k+1)} \odot \mathbf{g}'(\mathbf{z}^{(k)}) \text{ of dimension } d_k \times 1$$

- Gradient matrix at layer k is then

$$\begin{aligned}\frac{\partial e}{\partial \mathbf{W}^{(k)}} &= \frac{\partial e}{\partial \mathbf{z}^{(k)}} \frac{\partial \mathbf{z}^{(k)}}{\partial \mathbf{W}^{(k)}} \\ &= \mathbf{d}^{(k)} \cdot \mathbf{y}^{(k-1)T} \text{ of dimension } d_k \times d_{k-1} \\ \frac{\partial e}{\partial \mathbf{b}^{(k)}} &= \mathbf{d}^{(k)} \text{ of dimension } d_k \times 1\end{aligned}$$

Example

- 3 input nodes to 4 hidden nodes to 2 output nodes

Forward Quantity	Dimensions
$\mathbf{y}^{(0)}$	3×1
$\mathbf{W}^{(1)}$	3×4
$\mathbf{b}^{(1)}$	4×1
$\mathbf{z}^{(1)}$	4×1
$\mathbf{y}^{(1)}$	4×1
$\mathbf{W}^{(2)}$	4×2
$\mathbf{b}^{(2)}$	2×1
$\mathbf{z}^{(2)}$	2×1
$\mathbf{y}^{(2)}$	2×1
$\mathbf{t}$	2×1

Backward Quantity	Dimensions
$\mathbf{d}^{(2)}$	2×1
$\mathbf{d}^{(1)}$	4×1
$\partial e / \partial \mathbf{W}^{(2)}$	4×2
$\partial e / \partial \mathbf{b}^{(2)}$	2×1
$\partial e / \partial \mathbf{W}^{(1)}$	3×4
$\partial e / \partial \mathbf{b}^{(1)}$	4×1